Computational Physics Lectures

Week 00: Object-oriented Python

Bjørk Hammer
Department of Physics and Astronomy
Aarhus University
Denmark

Contents

0 Object-oriented Python **3**

0.1 Introduction . . . 3
0.2 Classes and objects . . . 3
0.3 Methods I . . . 4
0.4 Built-in methods . . . 5
0.5 Object attributes . . . 5
0.6 Object initialization . . . 6
0.7 Object comparison . . . 6
0.8 Methods II . . . 7
0.9 Attribute or method? . . . 9
0.10 Protected attributes and methods . . . 10
0.11 Encapsulation . . . 11
0.12 Encapsulation: Full example . . . 12
0.13 Polymorphism . . . 14
0.14 Inheritance . . . 14
0.15 Method overriding . . . 16
0.16 Class attributes . . . 18
0.17 Instance, static, and class methods . . . 18

© Bjørk Hammer

August 23, 2026

Week 0

Object-oriented Python

0.1 Introduction

Much programming in a computer language like *Python* is about handling data. Often, when writing such programs, a programmer wants to be able to:

- reuse parts of the code when data or operations on it take a new form, rewriting only the necessary parts of the code
- shield parts of the code (e.g. variables and functions) from other parts of the code to help collaborators assess the code in self-contained chunks
- develop a higher level of abstraction when handling the data, not having to care everywhere in your code about how exactly data is represented

These objectives may be achieved in *Object-Oriented Programming* in which the data is referred to by object variables that are constructed in such a way that *"they know by themselves how operations are made on the data"*.

In the following sections, it is shown how this is done in *Python*.

0.2 Classes and objects

A *Python class* is a means to store data in some organized manner. The simplest possible declaration of a class, one that contains no data looks like this:

```
class EmptyClass():
    pass
```

The class has the name `EmptyClass` and does nothing because of the `pass` statement. Instances of a class are called *Python objects*. They are created by calling the class as if it were a function. Here two objects, two instances of class `EmptyClass`, are created and inspected with the `print` function:

```
object1 = EmptyClass()
print(object1)
object2 = EmptyClass()
print(object2)
```

```
<__main__.EmptyClass object at 0x11260dad0>
<__main__.EmptyClass object at 0x1127f1fd0>
```

By default, the `print` function reveals the memory location of the objects, and it is seen that the two empty objects are located at different positions in the memory. Asking if the two objects are the same:

```
print('object1 == object2:', object1 == object2)
```

```
object1 == object2: False
```

we get the answer that they *are not*, which is a little funny since they are both empty. We shall return to that below.

0.3 Methods I

The class declaration normally involves some definitions of functions, so-called *Python methods*, that act on the objects. As a first example, consider that a method `id_as_string()` is added to the `ClassWithMethod` class:

```
class ClassWithMethod():

    def id_as_string(self):
        return str(hex(id(self)))
```

The method takes one argument, `self`, which is a reference to the instance of the class (= the object). Now, making one such instance, we may call the method, by adding a `.` and the `id_as_string()` after the variable name, here `object1`, referring to the instance:

```
object1 = ClassWithMethod()
print('object1: ',object1)
object1.id_as_string()
```

```
object1:  <__main__.ClassWithMethod object at 0x11260da50>
'0x11260da50'
```

Note how the `id_as_string()` method is defined in the class declaration as if it takes an argument, `self` while we do not provide that argument when the method is invoked as in `object1.id_as_string()`. This comes about because *Python* is providing that argument for us behind the scenes. In fact, `object1.id_as_string()` carries enough information to provide the proper value for `self`, namely the object `object1` preceding the `.id_as_string()`. Thus *Python* does not need us to declare that first argument. It is a little funny, but you will get used to it. If you try to give it anyhow:

```
object1.id_as_string(object1)
```

You will get this error message:

```
TypeError: id_as_string() takes 1 positional argument but 2 were given
```

that tells you that the method got two arguments, which are in fact the `object1` that *Python* provided from `object1.` and the `object1` that you gave in `.id_as_string(object1)`.

So far, our use of a class and some objects being instances of that class looks ugly, but don't despair, in a minute it will appear highly logical and useful.

0.4 Built-in methods

It turns out that when the `print` function refers to a *Python object*, it calls a method called `__str__()` on the object. I.e. the `<__main__.ClassWithMethod object at 0x11260da50>` output is the result of the `print` function performing a `object1.__str__()` evaluation. We may alter that very method in the class definition e.g. like this:

```
class SelfPrintingClass():

    def __str__(self):
        return 'an instance of the SelfPrintingClass class at ' + str(hex(id
   (self)))

object1 = SelfPrintingClass()
print(object1)
object2 = SelfPrintingClass()
print(object2)
```

```
an instance of the SelfPrintingClass class at 0x11281d890
an instance of the SelfPrintingClass class at 0x11260dad0
```

Notice how the former output `<__main__.ClassWithMethod object at 0x11260da50>` has gone and our new layout has now been adopted. Albeit we have no data yet, we get a glimpse of the idea behind object oriented programming. The two variables, `object1` and `object2`, soon to contain some data, *do know how to present themselves when printed*. I.e. we have stowed away the problem of how some data is printed and our main program can be ignorant about that. In a minute we will have some real data to print, and this statement may become clearer.

0.5 Object attributes

Okay, time is up for adding some data to objects. Data stored as part of an object are called *attributes*. They can be referred to as `object_name` + `.` + `attribute_name`. Consider an empty class:

```
class EmptyClass():
      pass
```

Some instances of the class, i.e. some objects, may be created, and some attributes set:

```
object1 = EmptyClass()
object2 = EmptyClass()
object1.data = [1, 2]
object1.more_data = 'hello'
object2.data = [0, 0]
```

Later, the values of the attributes may be retrieved:

```
print(object1.data, object1.more_data, object2.data)
for object in [object1, object2]:
    print('    printed in for-loop:',object.data)
```

```
[1, 2] hello [0, 0]
    printed in for-loop: [1, 2]
    printed in for-loop: [0, 0]
```

Note how the same attribute, `data`, is added to each of the two objects, but with different values. This reflects that the two objects are indeed different instances that reside in different memory positions, as discussed previously.

0.6 Object initialization

Now, the whole idea of objects was to wrap data in clever data structures. With the above we are not yet obtaining much of that. For instance, the programmer interacting with our class still has to know the name of the object attribute to initiate the data. To get one step further, we shall turn to the built-in constructor method, `__init__()`, which is called whenever a new object is created, i.e. whenever a new instance of a class is made. The method takes `self` as input followed by any sequence of arguments, possibly with defaults. It may look like this:

```
class ClassWithInit():

    def __init__(self,tag='no tag'):
        self.tag = tag

    def __str__(self):
        return 'an instance of the ' + self.__class__.__name__ + \
            ' class with tag="' + self.tag + '"'

object1 = ClassWithInit() # init with no argument, expect 'no tag'
print(object1)
object2 = ClassWithInit() # init with no argument
print(object2)
object3 = ClassWithInit('special tag') # init with an argument
print(object3)
```

```
an instance of the ClassWithInit class with tag="no tag"
an instance of the ClassWithInit class with tag="no tag"
an instance of the ClassWithInit class with tag="special tag"
```

Now, the assignment of values to the object attribute happens in the `__init__()` method. Here, it receives no argument for the first two objects and one for the last object. So, the first two objects get created with the default value for the tag, while the last object gets a tag passed in the creation.

0.7 Object comparison

Returning to the question of whether or not two objects are the same we may test for the objects just created:

```
print('object1 == object2:', object1 == object2)
print('object1 == object3:', object1 == object3)
```

```
object1 == object2: False
object1 == object3: False
```

and find that even the first two, `object1` and `object2`, that have similar attribute values (the default `'no tag'`) are not considered the same. The situation can be changed by modifying the built-in method, `__eq__()`, on the class like this:

```
class ClassWithEqual():

    def __init__(self,tag='no tag'):
        self.tag = tag

    def __str__(self):
        return 'tag="' + self.tag + '" at: ' + str(hex(id(self)))

    def __eq__(self, other):
        if isinstance(other, self.__class__):
            # they are of the same class, so result depends on the tag
            return self.tag == other.tag
        else:
            # they are not even of the same class, so cannot be equal
            return False

object1 = ClassWithEqual()
print(object1)
object2 = ClassWithEqual()
print(object2)
object3 = ClassWithEqual('special tag')
print(object3)
```

```
tag="no tag" at: 0x112827450
tag="no tag" at: 0x11260d810
tag="special tag" at: 0x11281d890
```

Now the comparison gives the expected:

```
print('object1 == object2:', object1 == object2)
print('object1 == object3:', object1 == object3)
```

```
object1 == object2: True
object1 == object3: False
```

0.8 Methods II

We now know enough about classes to make a non-trivial one. We shall define class `Rotor` as one containing and handling lists that contain more than one element. The class has a method, `rotate_data()` which makes a cyclic shift of the elements in the list:

```
class Rotor():

    def __init__(self, some_data):
```

```
        assert isinstance(some_data,list) and len(some_data) > 1, \
                    'ERROR: invalid data, must be list with len > 1'
        self.data = some_data

    def __str__(self):
        return '-'.join([str(d) for d in self.data])

    def rotate_data(self):
        self.data = self.data[1:] + [self.data[0]]
```

We start by making an instance of the class, an object. If we forget to provide the data we get an error message:

```
rotor1 = Rotor()
```

```
TypeError: __init__() missing 1 required positional argument: 'some_data'
```

Similarly, if the data has the wrong format, either like this:

```
rotor1 = Rotor('no list')
```

or this:

```
rotor1 = Rotor([25])
```

we get our own error message:

```
AssertionError: ERROR: invalid data, must be list with len > 1
```

Let us now do it right:

```
rotor1 = Rotor([3, 5, 1])
print('rotor1: ',rotor1)
```

```
rotor1:  3-5-1
```

With a proper instance `rotor1` of the `Rotor` class, we may now call the `rotate_data()` method:

```
print('rotor1: ',rotor1)
rotor1.rotate_data()
print('rotor1: ',rotor1)
```

```
rotor1:  3-5-1
rotor1:  5-1-3
```

We may repeat the process on another instance of the class:

```
rotor2 = Rotor([2, 4, 1])
print('rotor2: ',rotor2)
rotor2.rotate_data()
print('rotor2: ',rotor2)
```

```
rotor2:  2-4-1
rotor2:  4-1-2
```

0.9 Attribute or method?

Here is an example of a class that recalculates a dependent piece of data, an average of some other data, upon initialization:

```
class ClassWithAttribute():

    def __init__(self,a,b):
        self.a = a
        self.b = b
        self.average = (a + b)/ 2

    def __str__(self):
        return 'a={}, b={}, average={}'.format(self.a,self.b,self.average)
```

```
object1 = ClassWithAttribute(1,3)
object2 = ClassWithAttribute(5,7)
print(object1)
print(object2)
object1.average  # note: no () since it is a value
```

```
a=1, b=3, average=2.0
a=5, b=7, average=6.0
2.0
```

The class can be rewritten slightly, so that the dependent piece of data (in this case the average of the data received upon initialization) is available via a *method* on the object:

```
class ClassWithMethod():

    def __init__(self,c,d=7):
        self.c = c
        self.d = d

    def average(self):
        return (self.c + self.d) / 2

    def __str__(self):
        return 'c={}, d={}, average={}'.format(self.c,self.d,self.average())
```

```
object1 = ClassWithMethod(1,3)
object2 = ClassWithMethod(5)   # note: 7 will be assumed for d
print(object1)
print(object2)
object1.average()  # note: () since it is a method
```

```
c=1, d=3, average=2.0
c=5, d=7, average=6.0
2.0
```

Notice how `()` must be given on `.average` since it is now a method (= a function) rather than just an attribute.

We can get rid of the annoying parentheses, `()`, by introducing the decorator `@property`:

```
class ClassWithPropertyDecoratedMethod():
```

```
    def __init__(self,e,f=None):
        self.e = e

        if f is not None:
            self.f = f
        else:
            # assume same value twice
            self.f = e

    @property
    def average(self):
        return (self.e + self.f) / 2

    def __str__(self):
        return 'e={}, f={}, average={}'.format(self.e,self.f,self.average)
```

```
object1 = ClassWithPropertyDecoratedMethod(1,3)
object2 = ClassWithPropertyDecoratedMethod(5)   # note: same value will be
    assumed for f
print(object1)
print(object2)
object1.average  # note: no () since it is a property-decorated method
```

```
e=1, f=3, average=2.0
e=5, f=5, average=5.0
2.0
```

where now `object1.average` is invoked without the `()` parentheses, which is mainly a question of convenience for the user of the class. The take-home here is that on objects, attributes that appear to be variables might in fact be methods that act on the data. This was precisely the motivation for introducing object-oriented programming, namely that once a class instance (= an object) is created with some data, the object knows how to manipulate the data, in this case how to evaluate a mean value.

0.10 Protected attributes and methods

Sometimes the creator of a class does not intend the user of the class to use some of the defined attributes or methods. Whenever that is the case, the attributes or methods will be given a leading underscore character, `_`.

```
class ClassWithProtection():

    def __init__(self,c,d=7):
        self._c = c
        self._d = d

    def _average(self):
        return (self._c + self._d)/ 2

    def __str__(self):
        return 'c={}, d={}, average={}'.format(self._c,self._d,self._average
    ())
```

Now instantiating and printing `ClassWithMethod` and `ClassWithProtection` looks identical:

```
object1 = ClassWithMethod(1,3)
object2 = ClassWithProtection(1,3)
print(object1)
print(object2)
```

```
c=1, d=3, average=2.0
c=1, d=3, average=2.0
```

but while the attributes and the method on the object of type `ClassWithMethod` are available:

```
object1.c, object1.d, object1.average()
```

```
(1, 3, 2.0)
```

those on the object of type `ClassWithProtection` are hidden:

```
object2.c, object2.d, object2.average()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[7], line 1
----> 1 object2.c, object2.d, object2.average()

AttributeError: 'ClassWithProtection' object has no attribute 'c'
```

Obviously, they are not hidden to any higher degree than that they can be revealed if the underscores are included:

```
object2._c, object2._d, object2._average()
```

```
(1, 3, 2.0)
```

but convention is that the user should never access attributes or methods with leading underscores in their names.

0.11 Encapsulation

In the above example for the `ClassWithAttribute` class there is one problem. Consider we have an instance of the class:

```
object = ClassWithAttribute(1,3)
print(object)
```

```
a=1, b=3, average=2.0
```

If one now changes one of the attributes, `.a` or `.b` on the object, the `.average` attribute is not recalculated and will be wrong:

```
object.a = 100
print(object)
```

```
a=100, b=3, average=2.0
```

Obviously, this problem will not be there with the two other classes, `ClassWithMethod` and `ClassWithPropertyDecoratedMethod`, but for these classes, the average value is recalculated every time it is accessed, which would be a waste had it been a more complicated computation, say the simulation of a physics problem.

The way to resolve these issues is called *encapsulation*. It acts by "hiding" the data in the object and introducing an interface for the external world to access it. Following the convention used for protected attributes, one would prepend an underscore to the attribute name of a hidden variable, i.e. one would turn `.g` into `._g`. Then one would define a property-decorated function `.g` that returns the value of the hidden attribute. This function is called a *getter-function*. We have something like this:

```
class ClassName():
    def __init__(self,g):
        self._g = g

    @property
    def g(self):
        return self._g
```

Now for the outside world to be able to change the value of the data in the object, one then introduces a so-called *setter-function*:

```
class ClassName():
    def __init__(self,g):
        self._g = g

    @property
    def g(self):
        return self._g

    @g.setter
    def g(self,new_g):
        self._g = new_g
```

where the decorator, `@g.setter`, is an important part of the syntax. Here the only thing that happens when an attribute is changed is that the hidden attribute is updated. However, now the object may detect that some of its data has changed and make a note of it. I.e. the `def g(self, new_g)` could be expanded as:

```
    @g.setter
    def g(self,new_g):
        self._g = new_g
        self.something_happened = True
```

and then the class may react to this in its other methods. Below is given an example of that.

0.12 Encapsulation: Full example

Here is a full example, where the data of a class are shielded off from the outside world with getter- and setter-functions, and where the calculation of some property is delayed until the point, where it is requested.

```
class LazyCalculation():
    def __init__(self,g,h):
        self._g = g
        self._h = h
        self.average_calculated = False

    def __str__(self):
        return 'g={}, h={}, average={}'.format(self._g,self._h,self.average)

    @property
    def g(self):
        return self._g

    @property
    def h(self):
        return self._h

    @g.setter
    def g(self,new_g):
        print('... changing g')
        self._g = new_g
        self.average_calculated = False

    @h.setter
    def h(self,new_h):
        print('... changing h')
        self._h = new_h
        self.average_calculated = False

    @property
    def average(self):
        if not self.average_calculated:
            print('... calculating')
            self._average = (self._g + self._h) / 2
            self.average_calculated = True
        return self._average
```

```
object = LazyCalculation(1,3)
print(object)
object.average
object.g = 100
object.h = 300
object.average
```

```
... calculating
g=1, h=3, average=2.0
... changing g
... changing h
... calculating
200.0
```

Note how the average was not calculated between changing `.g` and `.h`, but only when the `.average` attribute was accessed at the very end.

0.13 Polymorphism

Polymorphism means that objects of different classes can be used through the same interface. The calling code does not need to know an object's exact class; it only needs the object to provide the required operation. Thus the same method call can produce class-specific behaviour for different kinds of data.

In the example below, both `OneList` and `TwoNumbers` provide an `average()` method, although they store their data differently. The loop can therefore treat both objects alike.

```
class OneList:
    def __init__(self, the_list):
        self.list = the_list

    def average(self):
        return sum(self.list) / len(self.list)

class TwoNumbers:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def average(self):
        return (self.a + self.b) / 2
```

```
one_list = OneList([1, 2, 3, 4])
two_numbers = TwoNumbers(2, 8)

objects = [one_list, two_numbers]
for object in objects:
    print(object.average())       # <----- Polymorphism at play!
```

```
2.5
5.0
```

0.14 Inheritance

Inheritance defines a new class from an existing class. The existing class is called the *base* or *parent* class, while the new class is called the *derived* or *child* class. A child inherits the parent's methods and attributes and may add or specialize behaviour. Inheritance is useful when classes share a genuine common interface and implementation.

Below, `Distribution` stores the interval and implements the common `plot()` method. `Uniform` and `Normal` inherit that functionality, and implement the `_set_xvals()` method that provides random x-values according to either a uniform or a normal distribution.

```
import numpy as np
import matplotlib.pyplot as plt

class Distribution:

    def __init__(self, xmin=0, xmax=1):
        self.xmin = xmin
```

```
        self.xmax = xmax
        self._set_xvals()

    def plot(self, ax):
        Nbins = 12
        assert Nbins > 1, 'Nbins must be at least 2'
        xedges = np.linspace(self.xmin, self.xmax, Nbins+1)
        xcenters = (xedges[:-1] + xedges[1:]) / 2
        xwidth = xedges[1] - xedges[0]
        bars, _ = np.histogram(self.xvals, xedges)
        P = bars / np.sum(bars) / xwidth
        ax.bar(xcenters, P, width=xwidth, facecolor='C0', alpha=0.25)
        ax.bar(xcenters, P, width=xwidth, facecolor='None', edgecolor='k')
        ax.set_title(self.__class__.__name__)

class Uniform(Distribution):

    def _set_xvals(self):
        self.xvals = np.random.rand(10000) * (self.xmax - self.xmin) \
                                                    + self.xmin

class Normal(Distribution):

    def _set_xvals(self):
        self.xvals = np.random.randn(10000)

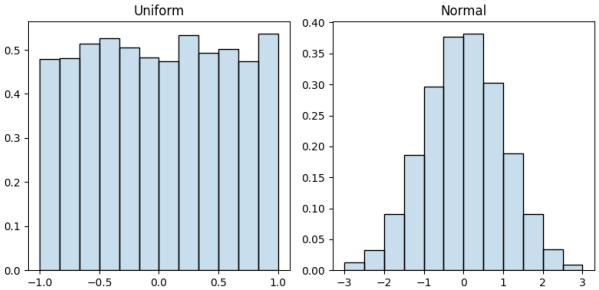
fig, axes = plt.subplots(1, 2, figsize=(8, 4))

uniform_distribution = Uniform(-1)
normal_distribution = Normal(xmin=-3, xmax=3)

distributions = [uniform_distribution, normal_distribution]

for distribution, ax in zip(distributions, axes):
    distribution.plot(ax)

fig.tight_layout()
fig.savefig('filename.png')
```



0.15 Method overriding

When a child class defines a method with the same name as a method in its parent class, the child's version is used for instances of the child class. This is called *method overriding*. The child may completely replace the inherited behaviour, or it may call the parent implementation through `super()` and extend it.

```
class Distribution:

    def __init__(self, xmin=0, xmax=1):
        self.xmin = xmin
        self.xmax = xmax
        self._set_xvals()

    def plot(self, ax):
        # Unchanged
        pass

    def _set_xvals(self): # unlike before the parent class has this method !
        self.xvals = np.array([])

class Uniform(Distribution):

    def _set_xvals(self):
        self.xvals = np.random.rand(10000) * (self.xmax - self.xmin) \
                                                    + self.xmin

class Normal(Distribution):

    def _set_xvals(self):
        self.xvals = np.random.randn(10000)
```

In this example, the parent class, `Distribution`, now has the `self._set_xvals()` method and would call it during initialization. Because `_set_xvals()` is overridden in the child classes, the method actually executed depends on the class of the object being created. Thus a `Uniform` object receives uniformly distributed values, while a `Normal` object receives normally distributed values.

The calling code would be as in the above section on polymorphism, meaning that it again uses one common interface, while the object's actual class determines which implementation of `_set_xvals()` is executed. Method overriding is therefore one of the mechanisms that enables polymorphism.

Method overriding can also be applied to the constructor method, `__init__()`. In that case, care must be taken to call the constructor of the parent class if its initialization is still needed. The parent constructor may be accessed through `super()`:

```
class Distribution:

    def __init__(self, xmin=0, xmax=1):
        self.xmin = xmin
        self.xmax = xmax

    def plot(self, ax):
        # Unchanged
```

```
class Uniform(Distribution):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)  # <-- calling the constructor
                                           #     method on the parent class
                                           #     after this, self.xmin
                                           #     and self.xmax are well
                                           #     defined
        self.xvals = np.random.rand(10000) * (self.xmax - self.xmin) \
                                                + self.xmin
class Normal(Distribution):

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)  # <-- calling the constructor
                                           #     method on the parent class
        self.xvals = np.random.randn(10000)
```

Note how arguments, `*args`, and key-value arguments, `**kwargs`, are passed from the constructors of the child classes to that of the parent class. This means that the default values only need to be defined in the parent class and that the interval can be assumed well-defined once the child classes initiate the distributions they implement.

The child classes may expect input arguments themselves, e.g. like here:

```
class Normal(Distribution):

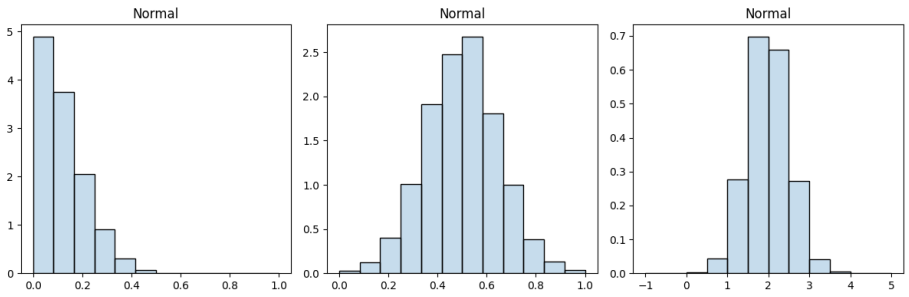
    def __init__(self, sigma, *args, center=0, **kwargs): # <--- sigma,
                                                          #    and center
        super().__init__(*args, **kwargs)
        self.xvals = sigma * np.random.randn(10000) + center
```

Which can then be instantiated and plotted in various ways:

```
normal1 = Normal(0.15)
normal2 = Normal(0.15, center=0.5)
normal3 = Normal(0.5, -1, center=2, xmax=5)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

normals = [normal1, normal2, normal3]
for normal,ax in zip(normals, axes):
    normal.plot(ax)
```



0.16 Class attributes

A class attribute belongs to the class and is shared by its instances unless an instance shadows it with an attribute of the same name. It is defined in the class body outside the methods. An instance attribute is normally assigned through `self` and may differ from one object to another.

```
class Measurement:
    unit = 'm'                # class attribute
    number_created = 0        # class attribute

    def __init__(self, value):
        self.value = value    # instance attribute
        Measurement.number_created += 1

first = Measurement(2.5)
second = Measurement(4.0)
print(first.value, second.value)
print(first.unit, second.unit)
print(Measurement.number_created)

Measurement.unit = 'cm'         # changing a class attribute
print(first.unit, second.unit)
```

```
2.5 4.0
m m
2
cm cm
```

Note the usage of the class name, `Measurement`, when modifying a class attribute.

0.17 Instance, static, and class methods

The methods we have seen so far are *instance methods*. They operate on a particular object and therefore receive `self` as their first argument.

Returning to the `TwoNumbers` class from above, we can add other types of methods:

```
class TwoNumbers:
    multiplication_factor = 1

    def __init__(self, a, b):
        self.a = a
        self.b = b

    def average(self):
        return (self.a + self.b) / 2

    @staticmethod
    def add(x, y):
        return x + y

    @classmethod
    def with_equal_numbers(cls, value):
        return cls(value, value)
```

The method `average()` is an *instance method*. It acts on the data stored in a particular object:

```
object = TwoNumbers(2, 8)
object.average()
```

```
5.0
```

The method `add()` is a *static method*. It adds two numbers and hence belongs logically to the class (as the class handles two numbers), but it neither receives `self` nor has access to the data of a particular object:

```
TwoNumbers.add(2, 8)
```

```
10
```

The method `with_equal_numbers()` is a *class method*. It receives the class itself through `cls` and is here used as an alternative constructor that takes only one argument and still returns a valid object:

```
object = TwoNumbers.with_equal_numbers(4)
print(object.a, object.b, object.average())
```

```
4 4 4.0
```

In summary, instance methods operate on an object through `self`, static methods are ordinary functions stored inside the class namespace, and class methods operate on the class itself through `cls`.